Riphah International University Lahore, Pakistan



Riphah School of Computing & Innovation

Final Year Project

PROJECT REPORT

SafarSync

Project Team

Student Name	Student ID	Program	Contact Number	Email Address
Muhammad Talha	46287	BS-CS	03164247480	46287@students.riphah.edu.pk

Mr. Khawar Iqbal
Senior Lecturer

Project Report

SafarSync

Change Record

Author(s)	Version	Date	Notes	Supervisor's Signature
Muhammad Talha	1.0	07-08-2025	Original Idea	
Muhammad Talha	1.1	08-08-2025	Live Location and Emergency Contact	
Muhammad Talha	1.2	23-10-2025	Sasta Tour, Virtual Tours, Wishlist, Filters	
Muhammad Talha	1.2	26-04-2026	Implementation	
			<Changes Based on Feedback from Supervisor>	

APPROVAL

PROJECT SUPERVISOR

Comments: _____

_____

Name:_____

Date:_____ Signature:_____

PROJECT MANAGER

Comments: _____

_____

Date:_____ Signature:_____

HEAD OF THE DEPARTMENT

Comments: _____

_____

Date:_____ Signature:_____

Dedication

To,

My Beloved Parents,

&

My supervisor

Acknowledgements

I am deeply grateful to Almighty Allah, the Most Merciful and Benevolent, whose countless blessings have guided my thoughts and strengthened my ambitions. I sincerely thank Him for granting me supportive teachers, loving parents, caring siblings, and wonderful friends. With great respect and devotion, I send my heartfelt praise to the Holy Prophet Muhammad (S.A.W), whose teachings have illuminated our faith and guided us toward righteousness.

I am profoundly thankful to my respected supervisor, whose wisdom, patience, and unwavering guidance have been instrumental throughout this journey. Their insightful feedback, timely encouragement, and belief in my potential have shaped not only this dissertation but also my growth as a researcher and learner. I am truly blessed to have had a mentor who exemplifies dedication and excellence. May Allah reward them abundantly for their support and kindness.

I owe my deepest gratitude to my beloved parents, whose unwavering prayers, moral and financial support, and endless encouragement have been a pillar of strength throughout my academic journey. Their love and sacrifices can never be fully repaid, and their guidance has always been a beacon of light in my life. The names of my parents will forever hold a special place in my heart, just as they do in my work. I would also like to extend my sincere thanks to everyone who contributed to the completion of this dissertation. Their support has made my academic journey truly memorable. May Allah bless them all with long, happy, and peaceful lives. Ameen.

Executive Summary

The **SafarSync App** is an innovative solution designed to empower both male/female and general users in Pakistan by providing inclusive access to travel support, job opportunities, skill learning, helpdesk services, and AI-powered mental health/chatbot assistance.

Traditional support systems are fragmented, outdated, or inaccessible, leaving marginalized communities without reliable resources. Existing digital platforms are either too generic or lack inclusivity. SafarSync addresses these limitations by offering a **modular, mobile-first ecosystem** that is secure, scalable, and user-friendly.

The app integrates multiple modules:

- **Blog** for community expression
- **Skill Learning** for tutorials and personal growth
- **Helpdesk** for urgent support and NGO connections
- **AI Chatbot** for travel guidance and mental health

Built on a **three-tier architecture** (Flutter frontend, Node.js backend, MySQL/Firebase data layer), SafarSync ensures modularity, maintainability, and real-time synchronization.

Key Features:

- Inclusive Dashboard with modular access
- Skill Learning with progress tracking
- Helpdesk with priority for transgender users
- AI Chatbot for travel + mental health
- Secure Authentication (MFA, Email Verification)
- Admin Panel for moderation and analytics

Table of Contents

Dedication....4
Acknowledgements....5
Executive Summary....6
Table of Contents....7
List of Figures....9
List of Tables....10
Chapter 1....111
Introduction....112
1.1. Background....112
1.2. Motivations and Challenges....112
1.3. Goals and Objectives....112
1.4. Literature Review/Existing Solutions....112
1.5. Gap Analysis....113
1.6. Proposed Solution....113
1.7. Project Plan....113
1.7.1. Work Breakdown Structure....113
1.7.2. Roles & Responsibility Matrix....114
1.7.3. Gantt Chart....114
1.8. Report Outline....114
Chapter 2....115
Software Requirement Specifications....17
2.1. Introduction....116
2.1.1. Purpose....116
2.1.2. Document Conventions....116
2.1.3. Intended Audience and Reading Suggestions....116
2.1.4. Product Scope....18
2.1.5. References....117
2.2. Overall Description....117
2.2.1. Product Perspective....117
2.2.2. Product Functions....19
2.2.3. User Classes and Characteristics....117
2.2.4. Operating Environment....118
2.2.5. Design and Implementation Constraints....20
2.2.6. User Documentation....20
2.2.7. Assumptions and Dependencies....20
2.3. External Interface Requirements....20
2.3.1. User Interfaces....20
2.3.2. Hardware Interfaces....21
2.3.3. Software Interfaces....21
2.3.4. Communications Interfaces....21
2.4. System Features....21
2.4.1. System Feature 1....21
2.4.1.1. Description and Priority....21
2.4.1.2. Stimulus/Response Sequences....21
2.4.1.3. Functional Requirements....22
2.4.2. System Feature 2....22
2.4.2.1. Description and Priority....22
2.4.2.2. Stimulus/Response Sequences....22
2.4.2.3. Functional Requirements....22
2.4.3. System Feature 3 (and so on)....23
2.5. Other Nonfunctional Requirements....25

2.5.1. Performance Requirements 25
2.5.2. Safety Requirements 25
2.5.3. Security Requirements 25
2.5.4. Software Quality Attributes 25
2.5.5. Business Rules 26
2.6. Other Requirements 26
Chapter 3 28
Use Case Analysis 29
3.1. Use Case Model 29
3.2. Fully Dressed Use Cases 30
Chapter 4 35
System Design 31
4.1. Architecture Diagram 36
4.2. Entity Relationship Diagram with data dictionary 38
4.3. Class Diagram 39
4.4. Sequence / Collaboration Diagram 39
4.5. Operation contracts 40
4.6. Activity Diagram 41
4.7. State Transition Diagram 42
4.8. Component Diagram 43
4.9. Deployment Diagram 43
Chapter 5 44
Implementation 45
5.1. Important Flow Control/Pseudo codes 45
5.2. Components, Libraries, Web Services and stubs 45
5.3. Deployment Environment 46
5.4. Tools and Techniques 46
5.5. Best Practices / Coding Standards 46
5.6. Version Control 47
Chapter 6 37
Testing and Implementation 45
Appendices 48
Appendix A: Information / Promotional Material 49
Reference and Bibliography 52
Index 54

List of Figures

1.1	Gantt Chart	5
3.1	Use Case Model	28
4.1	System Architecture Diagram	36
4.2	Entity Relationship Diagram	38
4.3	Class Diagram	39
4.4	Sequence Diagram	39
4.5	Activity Diagram	41
4.6	State Transition Diagram	42
4.7	Component Diagram	43
4.8	Deployment Diagram	43

List of Tables

1.1	Roles & Responsibility Matrix	5
2.1	User Classes and Characteristics	10
3.1	Fully Dress Use Case	31-34
4.6	Operation Contracts	40

Chapter 1

Introduction

Chapter 1: Introduction

1.1. Background

In Pakistan, access to inclusive digital platforms remains limited, particularly for marginalized communities such as transgender individuals. Traditional systems for job searching, skill development, and mental health support are fragmented, outdated, or inaccessible. This creates barriers to empowerment, employment, and social inclusion.

SafarSync is designed as a mobile-first, inclusive platform that integrates multiple services—blogs, job portals, skill learning, helpdesk, and AI-powered chatbot support—into a single ecosystem. By leveraging modern technologies such as Flutter, Node.js, MySQL, and AI APIs, SafarSync provides a scalable, secure, and user-friendly solution that bridges the gap between marginalized communities and mainstream opportunities.

1.2. Motivations and Challenges

Motivations:

- To empower transgender and general users with equal access to opportunities.
- To provide a safe, inclusive digital space for expression and support.
- To integrate multiple services into one modular, scalable platform.

Challenges:

1. Ensuring **real-time synchronization** of data across modules.
2. Designing a **UI/UX** that is inclusive, intuitive, and scalable.
3. Implementing **secure authentication** (MFA, email verification).
4. Bridging **technical and social gaps** by connecting users with NGOs and verified resources.

1.3. Goals and Objectives

- Develop a **modular dashboard** for Blogs, Skills, Helpdesk, and Chatbot.
- Provide **secure authentication** with MFA and email verification.
- Build a **scalable three-tier architecture** for maintainability.
- Deliver a **neon-themed, user-friendly interface** optimized for inclusivity.
- Integrate **AI chatbot** for travel guidance and mental health support.

1.4. Literature Review/Existing Solutions

- **Skill Platforms** (Coursera, Digiskills): Offer training but are not localized or community-driven.

- **Helpdesk Systems**: Existing solutions are generic and not tailored for vulnerable communities.
- **Chatbots**: Widely used in customer service but rarely adapted for mental health or inclusivity.

1.5. Gap Analysis

- Lack of **inclusive platforms** for transgender users.
- Fragmented access to **jobs, skills, and support services**.
- Absence of **AI-driven mental health support** in local apps.
- Limited **scalability and modularity** in existing solutions.

1.6. Proposed Solution

SafarSync offers a **unified, modular platform** that integrates:

- **Blog Module** for community expression.
- **Job Portal** for employment opportunities.
- **Skill Learning** for tutorials and growth.
- **Helpdesk** for urgent support and NGO connections.
- **AI Chatbot** for travel and mental health guidance.

The app uses a **three-tier architecture**:

- **Presentation Tier**: Flutter mobile app + React admin panel.
- **Application Tier**: Node.js + Express backend.
- **Data Tier**: MySQL + Firebase for real-time sync.

1.7. Project Plan

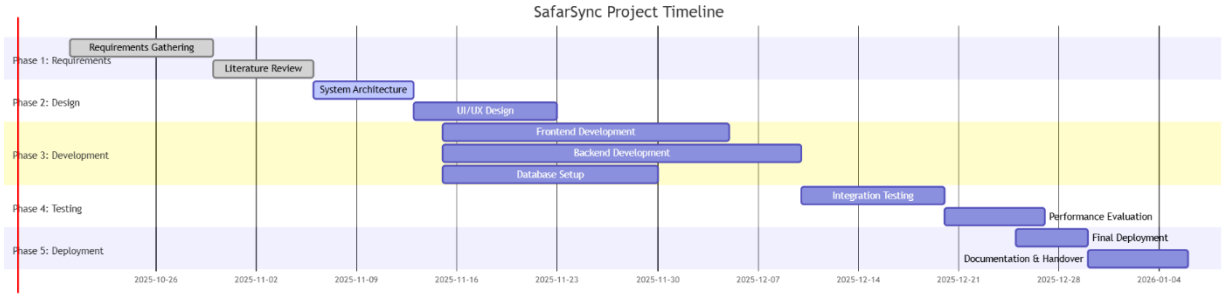
1.7.1. Work Breakdown Structure

- Requirement Gathering
- UI/UX Design (Figma)
- Frontend Development (Flutter)
- Backend Development (Node.js, Express)
- Database Setup (MySQL, Firebase)
- Module Integration (Blog, Jobs, Skills, Helpdesk, Chatbot)
- Testing & Debugging
- Deployment

1.7.2. Roles & Responsibility Matrix

Task	Name	Responsibility
Project Planning	Muhammad Talha	Timeline, WBS, Gantt Chart
UI/UX Design	Muhammad Talha	Figma design system, flows
Frontend Dev	Muhammad Talha	Flutter app screens
Backend Dev	Muhammad Talha	Node.js APIs, Sequelize ORM
Database	Muhammad Talha	MySQL schema, migrations
Testing	Muhammad Talha	Unit, integration, performance
Documentation	Muhammad Talha	Report writing, diagrams

1.7.3. Gantt Chart



1.8. Report Outline

This report is structured into seven chapters:

- **Chapter 1**: Introduction
- **Chapter 2**: Software Requirement Specifications
- **Chapter 3**: Use Case Analysis
- **Chapter 4**: System Design
- **Chapter 5**: Implementation
- **Chapter 6**: Testing & Evaluation
- **Chapter 7**: Conclusion & Future Work

Chapter 2

Software Requirement Specifications

Chapter 2: Software Requirement Specifications

2.1. Introduction

2.1.1. Purpose

The purpose of this document is to define the software requirements for SafarSync—a mobile and web-based platform designed to support both transgender and general users in Pakistan. The app provides access to travel support, job opportunities, skill development, helpdesk services, and AI-powered chatbot assistance. This SRS serves as a reference for developers, designers, testers, and stakeholders throughout the project lifecycle.

2.1.2. Document Conventions

- All modules are referenced using PascalCase (e.g., BlogModule, JobPortal).
- Functional requirements are labeled as FR-XX.
- Non-functional requirements are labeled as NFR-XX.
- Diagrams follow UML standards.

2.1.3. Intended Audience and Reading Suggestions

- **Developers**: For implementation of frontend, backend, and database logic.
- **Designers**: For UI/UX structure and component planning.
- **Testers**: For validating functional and non-functional requirements.
- **Supervisors/Reviewers**: For academic evaluation and feedback.

2.1.4. Product Scope

SafarSync is a modular, scalable application that integrates five core services:

- BlogModule
- SkillLearning
- HelpdeskSystem
- AIChatbot

It also includes secure authentication, an admin panel, and real-time data synchronization. The app is built using Flutter (mobile), React.js (admin), Node.js + Express (backend), and MySQL + Firebase (data layer).

2.1.5. References

- OpenAI API Documentation
- Firebase Realtime Database Docs
- Flutter & React.js Official Docs
- Pakistan Transgender Rights Bill (2018)

2.2. Overall Description

2.2.1. Product Perspective

SafarSync is a standalone application with modular integration. It communicates with external APIs (OpenAI, Firebase) and internal services (MySQL, Express controllers). The mobile app and admin panel share the same backend and database.

2.2.2. Product Functions

- **Authentication**: Login, Sign Up, Forgot Password, MFA
- **BlogModule**: View, create, edit blogs
- **SkillLearning**: Access tutorials, track progress
- **HelpdeskSystem**: Submit tickets, receive NGO responses
- **AIChatbot**: Travel and mental health support
- **AdminPanel**: Moderate content, manage users

2.2.3. User Classes and Characteristics

User Type	Description
General User	Access all modules, create profile
Transgender User	Priority helpdesk access, inclusive chatbot
Admin	Moderate blogs, review tickets, manage users

User Type	Description
NGO Representative	Respond to helpdesk tickets via admin panel

2.2.4. Operating Environment

- **Mobile App**: Android (Flutter)
- **Admin Panel**: Web (React.js)
- **Backend**: Node.js + Express
- **Database**: MySQL, Firebase
- **Hosting**: Firebase Hosting / Heroku / Vercel

2.2.5. Design and Implementation Constraints

- Must support both dark and light themes.
- Must comply with accessibility standards (WCAG 2.1).
- Must use secure password hashing (bcrypt).
- Must support modular API routing.

2.2.6. User Documentation

- User manual (PDF)
- In-app onboarding tutorial
- Admin panel guide

2.2.7. Assumptions and Dependencies

- Users have internet access.
- Firebase and OpenAI APIs remain available.
- Admins are trained to use moderation tools.

2.3. External Interface Requirements

2.3.1. User Interfaces

- Mobile app screens: Splash, Login, Dashboard, Blog, Jobs, Skills, Helpdesk, Chatbot, Profile
- Responsive design with neon-themed branding
- Bottom navigation bar for quick access

2.3.2. Hardware Interfaces

- Android mobile devices
- Web browsers (Chrome, Firefox, Edge)

2.3.3. Software Interfaces

- OpenAI API (Chatbot)
- Firebase (Realtime DB, Auth)
- MySQL (Relational DB)
- RESTful API (Node.js + Express)

2.3.4. Communications Interfaces

- HTTPS for secure data transmission
- WebSockets for real-time updates (optional)

2.4. System Features

2.4.1. Authentication Module

2.4.1.1. Description and Priority:

This module manages user login, sign-up, password reset, and multi-factor authentication. It is essential for secure access and identity verification. Priority: High Benefit: 9 Penalty: 9 Cost: 5 Risk: 8

2.4.1.2. Stimulus/Response Sequences:

User enters login credentials → System validates input → If correct, user is logged in; if incorrect, error message is shown. User clicks "Forgot Password" → System prompts for email → If valid, sends reset link; if invalid, shows error. User signs up → System checks for duplicate email → If unique,

account is created; if duplicate, error is shown. User enters OTP for MFA → System verifies code → If correct, access is granted; if expired or incorrect, retry prompt is shown.

2.4.1.3. Functional Requirements

REQ-AUTH-1: The system shall reject empty login or sign-up fields with a validation error.

REQ-AUTH-2: The system shall detect and block duplicate email registrations. **REQ-AUTH-3:** The system shall send a password reset link only to verified email addresses.

REQ-AUTH-4: The system shall validate OTP codes and handle expired or incorrect entries gracefully.

REQ-AUTH-5: The system shall log failed login attempts for security monitoring.

2.4.2. Blog Module

2.4.2.1. Description and Priority:

This module allows users to create, view, and comment on blog posts. It supports community engagement and expression. Priority: Medium Benefit: 7 Penalty: 4 Cost: 3 Risk: 5

2.4.2.2. Stimulus/Response Sequences:

User opens blog feed → System fetches posts → If successful, displays content; if failed, shows retry option. User submits a blog post → System validates content → If valid, saves post; if empty or invalid, shows error. User uploads media → System checks format → If supported, attaches media; if unsupported, shows error.

2.4.2.3. Functional Requirements:

REQ-BLOG-1: The system shall prevent submission of empty blog posts.

REQ-BLOG-2: The system shall validate media formats before upload.

REQ-BLOG-3: The system shall display fallback messages if blog content fails to load. **REQ-BLOG-4:** The system shall allow retrying failed blog submissions.

REQ-BLOG-5: The system shall log moderation actions and errors.

2.4.3. Skill Learning Module

2.4.3.1. Description and Priority:

This module provides access to tutorials and tracks user progress. It supports personal development. Priority: Medium Benefit: 7 Penalty: 5 Cost: 3 Risk: 4

2.4.3.2. Stimulus/Response Sequences:

User opens skill module → System loads categories → If successful, displays tutorials; if failed, shows error. User starts a tutorial → System plays video → If broken link, shows fallback message. User completes a lesson → System updates progress → If offline, stores locally and syncs later.

2.4.3.3. Functional Requirements:

REQ-SKILL-1: The system shall handle broken video links with fallback messages. **REQ-SKILL-2:** The system shall store progress locally during offline sessions.

REQ-SKILL-3: The system shall sync progress with Firebase when reconnected.

REQ-SKILL-4: The system shall validate tutorial metadata before display.

REQ-SKILL-5: The system shall notify users of missing or outdated content.

2.4.4. Helpdesk System

2.4.4.1. Description and Priority:

This module allows users to submit support tickets and receive responses. It prioritizes transgender users. Priority: High Benefit: 9 Penalty: 8 Cost: 5 Risk: 7

2.4.4.2. Stimulus/Response Sequences:

User opens helpdesk → System shows ticket form → If fields are empty, blocks submission. Transgender user submits ticket → System flags as priority → Routes to NGO dashboard. Admin responds → System updates ticket → Notifies user of response.

2.4.4.3. Functional Requirements:

REQ-HDS-1: The system shall validate ticket fields and reject empty submissions.

REQ-HDS-2: The system shall flag transgender tickets for priority routing.

REQ-HDS-3: The system shall notify users when a response is posted.

REQ-HDS-4: The system shall prevent duplicate tickets within a short time window.

REQ-HDS-5: The system shall log all ticket interactions for audit.

2.4.5. AI Chatbot

2.4.5.1. Description and Priority:

This module provides real-time travel and mental health support using AI. It adapts to user type. Priority: High Benefit: 8 Penalty: 7 Cost: 6 Risk: 6

2.4.5.2. Stimulus/Response Sequences:

User opens chatbot → System loads interface → If API fails, shows fallback message. User sends query → System processes via OpenAI → If malformed, prompts for clarification. User toggles bot type → System switches context → Responds accordingly**.**

2.4.5.3. Functional Requirements:

REQ-CHAT-1: The system shall handle API failures with fallback messages**.**

REQ-CHAT-2: The system shall prompt users to clarify ambiguous queries.

REQ-CHAT-3: The system shall toggle between general and transgender bot modes. **REQ-CHAT-4:** The system shall log chatbot interactions for review.

REQ-CHAT-5: The system shall prevent repeated queries during cooldown periods.

2.4.6. Admin Panel

2.4.6.1. Description and Priority:

This module allows admins to moderate content, manage users, and review tickets. Priority: Medium Benefit: 7 Penalty: 6 Cost: 4 Risk: 5

2.4.6.2. Stimulus/Response Sequences:

Admin logs in → System verifies role → If unauthorized, blocks access. Admin edits blog → System validates input → If invalid, shows error. Admin reviews ticket → System updates status → Logs action.

2.4.6.3. Functional Requirements:

REQ-ADMIN-1: The system shall enforce role-based access control.

REQ-ADMIN-2: The system shall validate moderation inputs.

REQ-ADMIN-3: The system shall log all admin actions.

REQ-ADMIN-4: The system shall notify admins of backend failures.

REQ-ADMIN-5: The system shall prevent unauthorized data edits.

2.5. Other Nonfunctional Requirements

2.5.1. Performance Requirements

- App must load within 3 seconds on 4G
- API response time < 500ms

2.5.2. Safety Requirements

- No sensitive data stored in plaintext
- Secure logout and session timeout

2.5.3. Security Requirements

- Passwords hashed with bcrypt
- JWT-based authentication

- Role-based access control

2.5.4. Software Quality Attributes

- Modularity
- Scalability
- Maintainability
- Accessibility

2.5.5. Business Rules

- Transgender users receive priority helpdesk routing
- Duplicate emails trigger error on sign-up
- Blogs must be approved by admin before publishing

2.6. Other Requirements

2.7.1 Database Requirements

- The system shall use **MySQL** as the primary relational database for storing user data, blog content, ticket records, and skill progress.
- The system shall use **Firebase Realtime Database** for real-time synchronization of helpdesk tickets and chatbot interactions.
- All sensitive data (e.g., passwords, user roles) shall be encrypted using industry-standard hashing algorithms (e.g., bcrypt).
- The database schema shall be normalized to at least **3NF** to ensure data integrity and reduce redundancy.
- Regular backups shall be scheduled to prevent data loss in case of system failure.

2.7.2 Internationalization and Localization

- The system shall support **English** as the default language.
- The architecture shall be designed to support **future localization** into Urdu and other regional languages.
- All user-facing strings shall be stored in a centralized configuration file to facilitate easy translation.

2.7.3 Legal and Ethical Requirements

- The system shall comply with **Pakistan's Transgender Persons (Protection of Rights) Act, 2018**, ensuring respectful and inclusive treatment of transgender users.
- User data shall be handled in accordance with **data privacy laws** and ethical standards.
- The system shall include a **terms of service** and **privacy policy** accessible from the login and profile screens.

- The AI chatbot shall include a disclaimer stating that it is not a substitute for professional mental health support.

2.7.4 Reuse and Modularity Objectives

- The system shall be developed using a **modular architecture**, allowing individual modules (e.g., Blog, Helpdesk, Chatbot) to be reused or extended in future projects.
- UI components shall be created as **reusable Figma components** and Flutter widgets.
- Backend routes shall follow **RESTful conventions** to ensure interoperability and future API reuse.

2.7.5 Accessibility Requirements

- The mobile app shall follow **WCAG 2.1** accessibility guidelines to ensure usability for users with visual or cognitive impairments.
- The app shall support **dark mode** and **high-contrast themes** for better readability.
- All interactive elements shall be keyboard-navigable and screen-reader friendly.

2.7.6 Deployment and Hosting

- The mobile app shall be deployed to the **Google Play Store**.
- The admin panel shall be hosted on **Firebase Hosting** or **Vercel**.
- The backend server shall be deployed on **Heroku** or a similar cloud platform with SSL support.

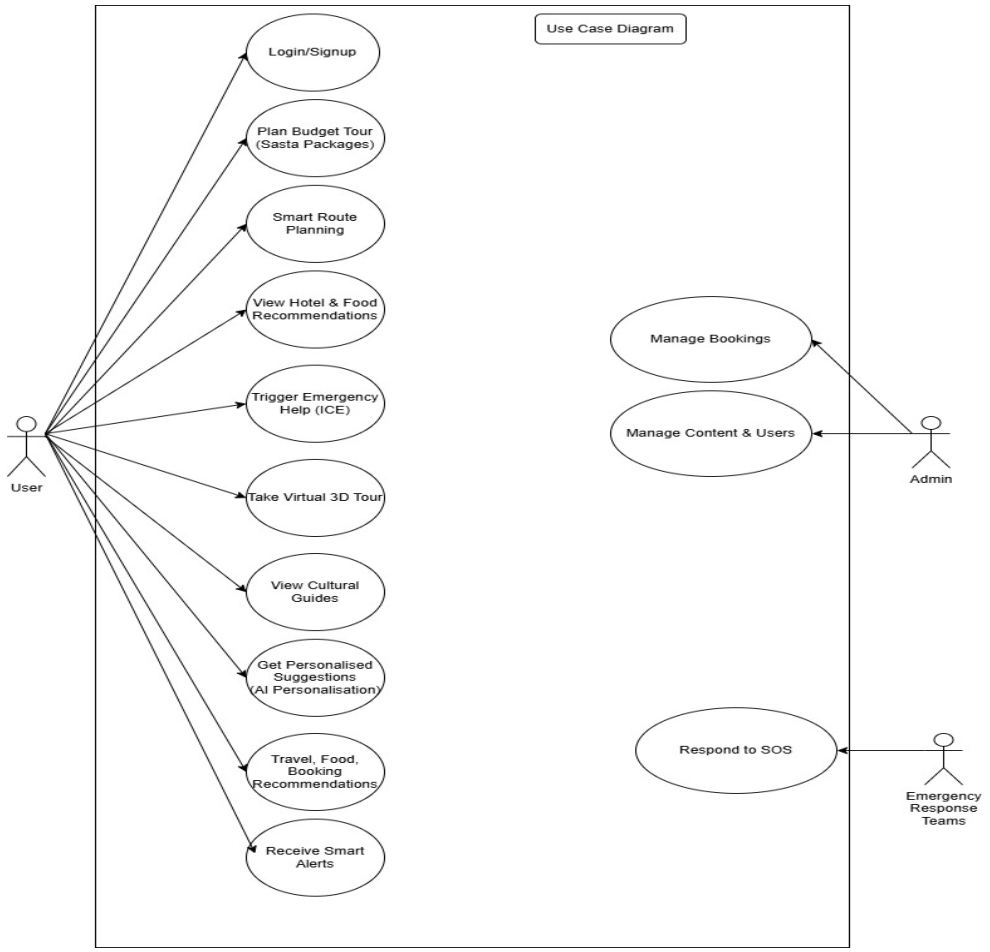
Chapter 3

Use Case Analysis

Chapter 3: System Analysis

The system allows a traveler to trigger an emergency alert through the mobile app. Once activated, the app captures the user's GPS location and sends it to the backend server. The backend generates a ticket and stores it in Firebase with a priority flag. Admins and NGO responders are immediately notified through the portal. Responders can then view, assign, and reply to the ticket, ensuring timely assistance. If the user is offline, the request is cached locally and synchronized once connectivity is restored. This ensures reliability, safety, and continuity of service even in critical scenarios.

3.1. Use Case Model



3.2. Fully Dressed Use Cases

Use Case: Login / Signup

Section	**Details**
Use Case Name	Login / Signup
Scope	SafarSync Mobile App
Level	User Goal

Section	Details
Primary Actor	User
Stakeholders	User, Admin
Preconditions	User has installed the app
Postconditions	User account created or authenticated
Success Guarantee	User gains access to dashboard
Minimal Guarantee	Error message displayed
Trigger	User taps login/signup
Main Success Scenario	1. User enters credentials 2. System validates 3. User redirected to dashboard
Extensions	Invalid credentials → error message Network failure → retry
Special Requirements	Secure password storage, OTP verification
Frequency of Use	High
Open Issues	Should social login be supported?

Use Case: Dashboard View

Section	Details
Use Case Name	View Dashboard
Scope	SafarSync Mobile App
Primary Actor	User
Preconditions	User logged in
Postconditions	Dashboard displayed
Main Success Scenario	1. User opens app 2. Dashboard shows trips, alerts, recommendations
Extensions	No trips → show empty state
Special Requirements	Responsive UI
Frequency of Use	High

Use Case: Map & Routing

Section	Details
Use Case Name	Plan Route
Scope	SafarSync Mobile App
Primary Actor	User
Preconditions	GPS enabled
Postconditions	Route generated
Main Success Scenario	1. User selects destination 2. System fetches weather & traffic 3. Route displayed

Section	Details
Extensions	No GPS → prompt user No internet → offline cache
Special Requirements	Real-time weather API
Frequency of Use	Medium

Use Case: Emergency Help

Section	Details
Use Case Name	Trigger Emergency Help
Scope	SafarSync Mobile App
Primary Actor	User
Stakeholders	NGO, Admin
Preconditions	User logged in
Postconditions	Ticket logged, responder notified
Main Success Scenario	1. User taps emergency 2. GPS sent 3. Ticket created 4. NGO notified
Extensions	Offline mode → store locally
Special Requirements	Response within 5 minutes
Frequency of Use	Rare but critical

Use Case: Virtual 3D Tour

Section	Details
Use Case Name	View Virtual Tour
Scope	SafarSync Mobile App
Primary Actor	User
Preconditions	Internet connection
Postconditions	Tour displayed
Main Success Scenario	1. User selects location 2. System loads 3D assets 3. Tour displayed
Extensions	Low bandwidth → fallback images
Special Requirements	VR compatibility
Frequency of Use	Medium

Use Case: Budget Planner

Section	Details
Use Case Name	Plan Budget Trip
Scope	SafarSync Mobile App
Primary Actor	User
Preconditions	User logged in
Postconditions	Budget plan generated
Main Success Scenario	1. User enters budget 2. System suggests hotels, transport 3. Plan displayed
Extensions	No matches → suggest nearest options
Special Requirements	Currency conversion
Frequency of Use	Medium

Use Case: Wishlist & Alerts

Section	Details
Use Case Name	Manage Wishlist & Alerts
Scope	SafarSync Mobile App
Primary Actor	User
Preconditions	User logged in
Postconditions	Wishlist updated, alerts configured
Main Success Scenario	1. User adds destination 2. System saves 3. Alerts triggered when deals/weather change
Extensions	No internet → store locally
Special Requirements	Push notifications
Frequency of Use	Medium

Use Case: AI Chatbot

Section	Details
Use Case Name	Use AI Chatbot
Scope	SafarSync Mobile App
Primary Actor	User
Preconditions	User logged in
Postconditions	Response displayed

Section	Details
Main Success Scenario	1. User asks query 2. Chatbot processes 3. Response displayed
Extensions	No internet → fallback FAQ
Special Requirements	Natural language support
Frequency of Use	High

Use Case: Admin Features

Section	Details
Use Case Name	Manage Portal
Scope	SafarSync Admin Panel
Primary Actor	Admin
Preconditions	Admin logged in
Postconditions	Portal updated
Main Success Scenario	1. Admin moderates content 2. Manages users 3. Updates packages
Extensions	Invalid data → error
Special Requirements	Role-based access
Frequency of Use	Medium

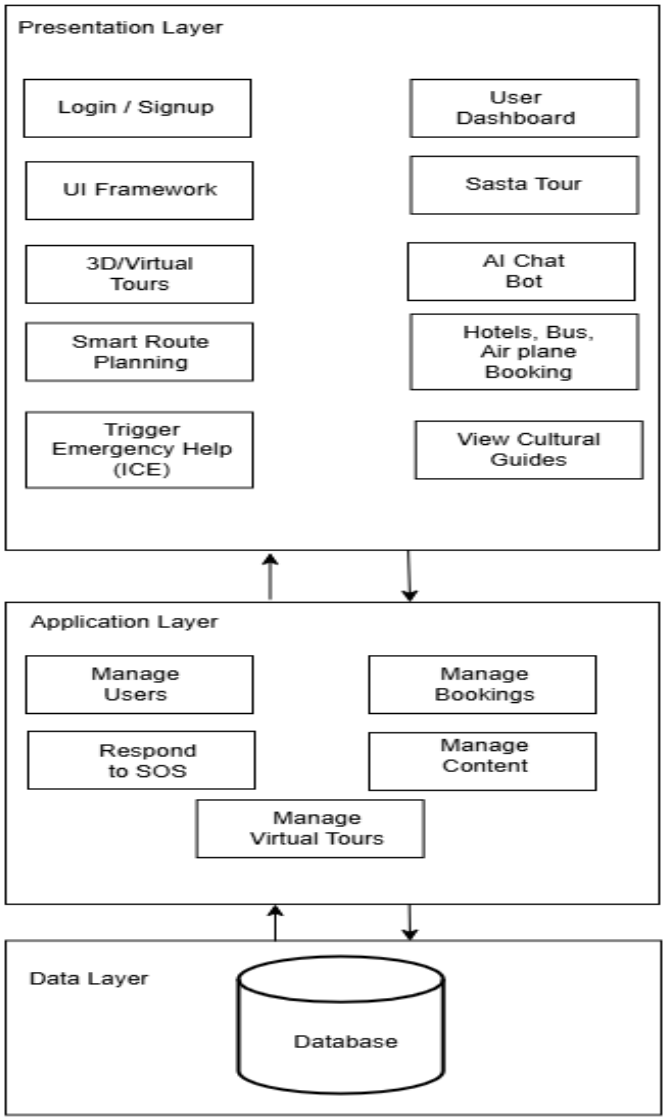
Chapter 4

System Design

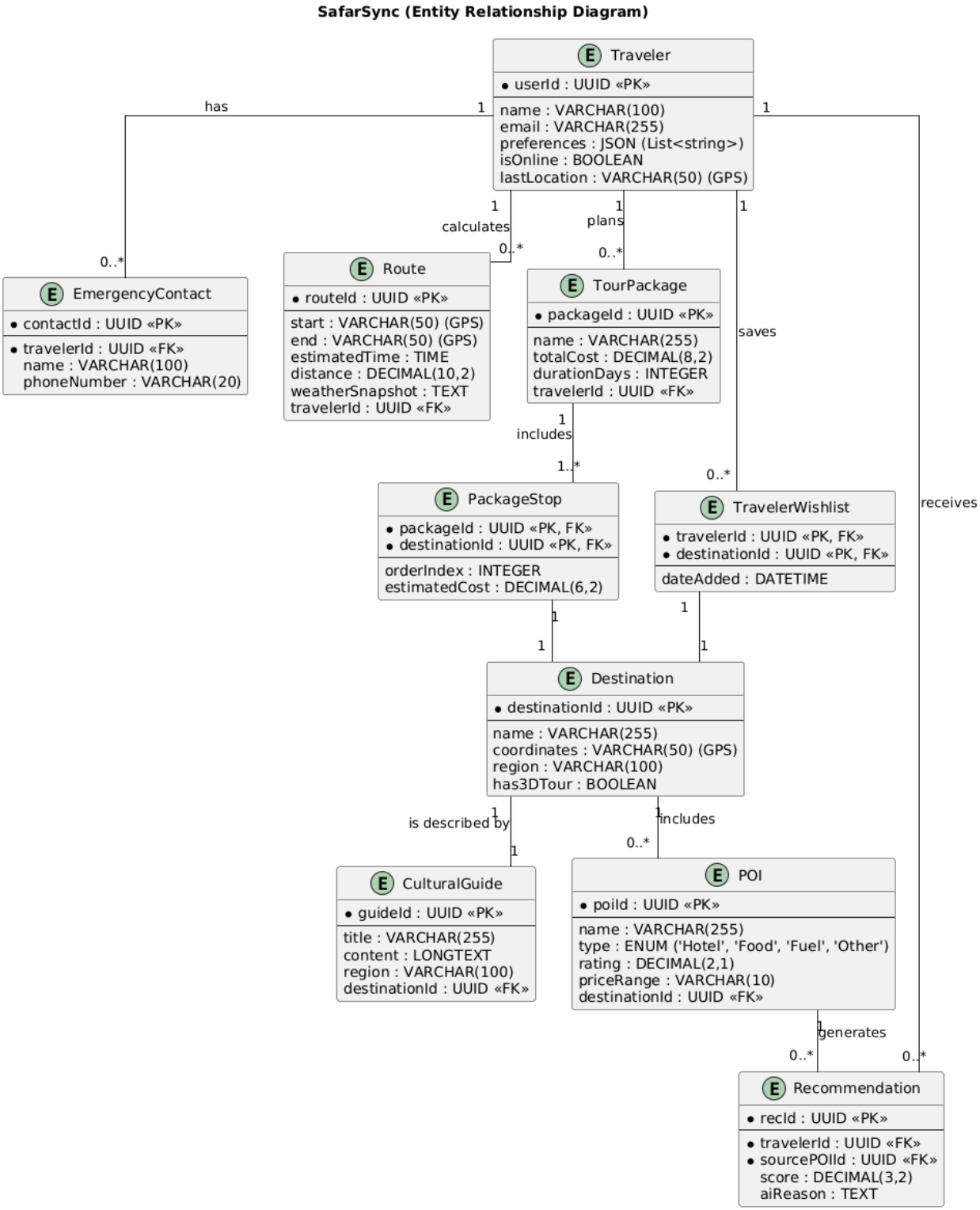
Chapter 4: System Design

SafarSync is a mobile-first travel assistant with an admin web portal. It combines trip planning, bookings, emergency support, AI-driven personalization, and immersive tours. The design emphasizes modular services, clear role-based access, offline resilience, and secure, observable operations.

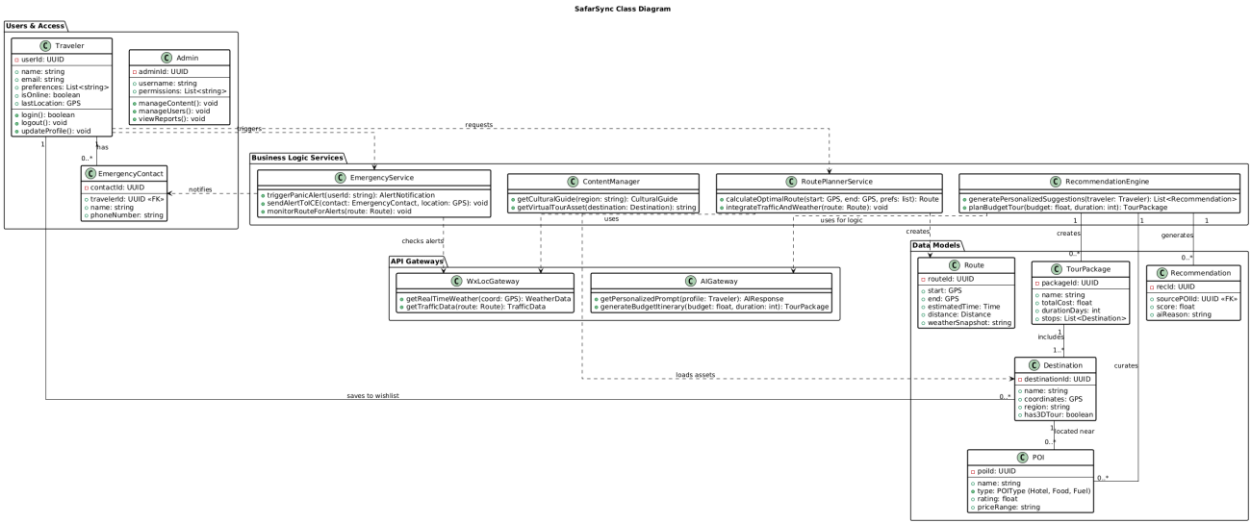
4.1. Architecture Diagram



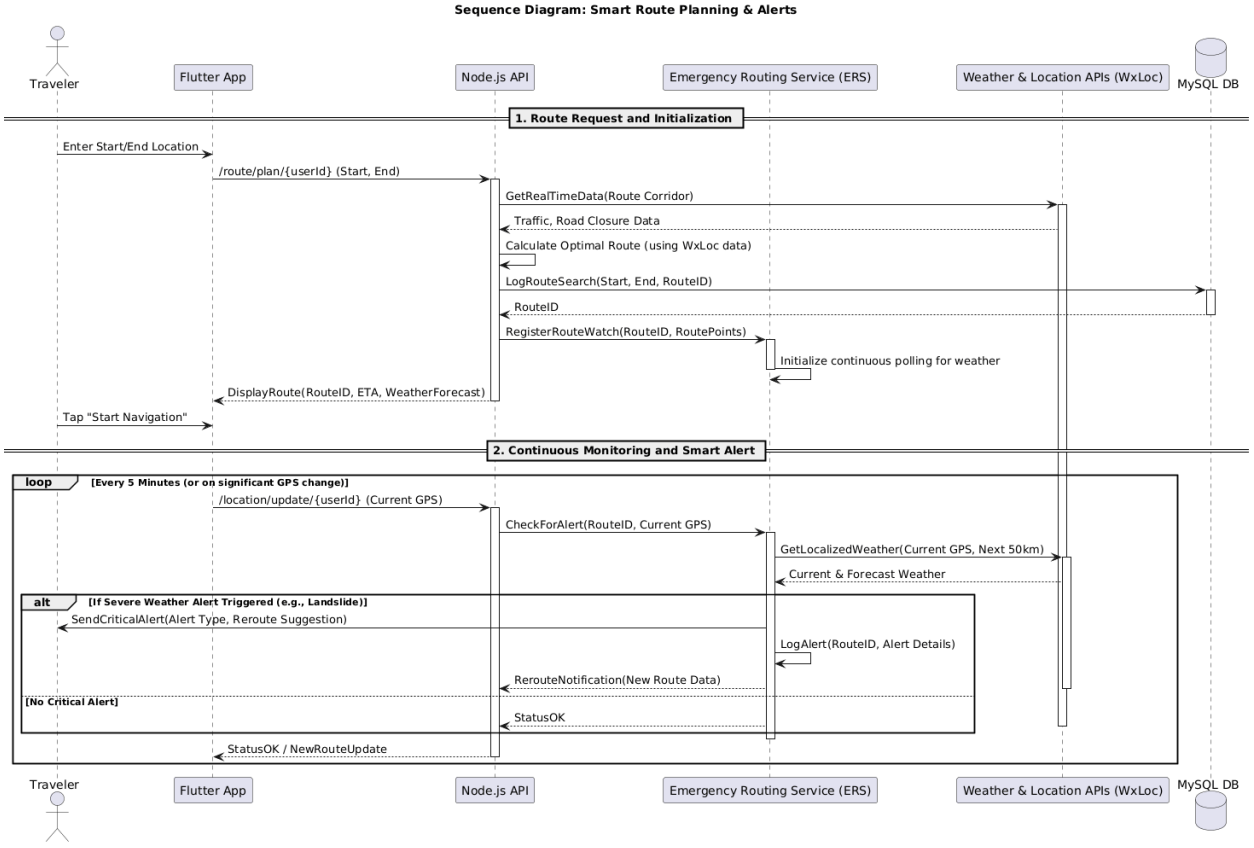
4.2. Entity Relationship Diagram with data dictionary



4.3. Class Diagram



4.4. Sequence / Collaboration Diagram



Chapter 5

Implementation

Chapter 5: Implementation

This chapter explains how the SafarSync system was implemented, covering the flow of control, pseudo codes, and the integration of different modules. It highlights the components, libraries, and web services used in development, along with the deployment environment and tools. The chapter also discusses coding standards, best practices, and version control strategies adopted to ensure reliability, scalability, and maintainability of the system. Overall, it provides a detailed technical insight into how the system was built and deployed.

5.1. Important Flow Control/Pseudo codes

The system follows modular flow control where each feature is encapsulated in its own service. For example, the **Emergency Help** flow begins with a user tapping the SOS button, which triggers GPS capture, ticket creation, and notification dispatch. Pseudo code for this flow:

```
IF user taps SOS button THEN
   capture GPS location
   create emergency ticket
   store ticket in Firebase
   notify Admin and NGO responders
ENDIF
```

5.2. Components, Libraries, Web Services and stubs

- **Frontend:** Flutter (mobile), React.js (admin portal)
- **Backend:** Node.js + Express
- **Database:** MySQL (structured data), Firebase (real-time notifications)
- **Libraries:** Material-UI, Sequelize ORM, JWT for authentication, Axios for API calls
- **Web Services:** Google Maps API, Weather API, Payment Gateway, OpenAI API for chatbot
- **Stubs:** Mock services for hotel and transport vendors during testing

5.3. Deployment Environment

The system is deployed in a cloud-based environment using AWS. Docker containers are used for packaging services, and Kubernetes manages scaling. The mobile app runs on Android devices, while the admin portal is accessible via modern browsers. Continuous integration and deployment pipelines ensure smooth updates.

5.4. Tools and Techniques

- **Development Tools:** VS Code, Postman, Figma for UI design
- **Techniques:** Agile methodology, modular design, RESTful APIs, responsive UI design
- **Testing Tools:** Jest, Mocha, Firebase Test Lab for mobile testing

5.5. Best Practices / Coding Standards

- Followed **SOLID principles** for maintainable code
- Used **CamelCase** naming convention
- Applied **linting tools (ESLint, Prettier)** for consistency
- Implemented **error handling and logging** for debugging
- Adopted **secure coding practices** (input validation, encryption, role-based access)

5.6. Version Control

Version control was managed using **Git and GitHub**. Branching strategies such as main, develop, and feature branches were used. Pull requests ensured peer review, and commits were tagged for releases. This allowed collaborative development and rollback in case of issues.

Chapter 6

Testing and Evaluation

Testing and Evaluation

6.1 Testing Approach

The testing approach will involve systematically checking all features of the project to ensure they work as expected. This will include both manual and automated testing. The goal is to identify and fix any bugs or issues before the project is released to users. Testing will be carried out in stages, starting with individual components and moving towards integrated testing of the whole system.

6.2 Test Plan

The test plan outlines how we will conduct our testing to ensure the project meets all requirements.

6.2.1 Features to be Tested

We will test the following features:

- User login, sign-up, password reset, and email verification.
- Tourist profile management, including viewing and updating profile information.
- Home page functionality, ensuring all sections display correctly.
- Places page, including searching, filtering, viewing details, and rating/reviewing places.
- Liking places and viewed places.
- Hotel page, including searching, filtering, viewing details, booking, and viewing bookings.
- Packages page, including searching, filtering, viewing details, and booking.
- Custom tour plan creation and PDF download.
- Email notifications for bookings and updates.
- Contact form functionality for tourists to reach out to hotel managers and tour operators.
- Static pages like "About Pakistan," slider, FAQs, Terms and Conditions, About Us, and Privacy Policy.
- Admin functionalities, including viewing and deleting places, recommended plans, hotels, and packages, and creating accounts for tour operators and hotel managers.
- Content manager functionalities, including managing places and recommended plans.
- Hotel manager functionalities, including managing hotels and viewing bookings.
- Tour operator functionalities, including managing packages.

6.3 Testing Tools and Environment

We will use a combination of manual testing and automated testing tools. The testing environment will mimic the production environment as closely as possible to ensure accurate results. Some of the tools we might use include:

- Real browsers.

- Browser DevTools.
- Postman for API testing.
- Different Devices.
- Internet Connection.
- Dummy Entries

6.4 List of Test Scenarios

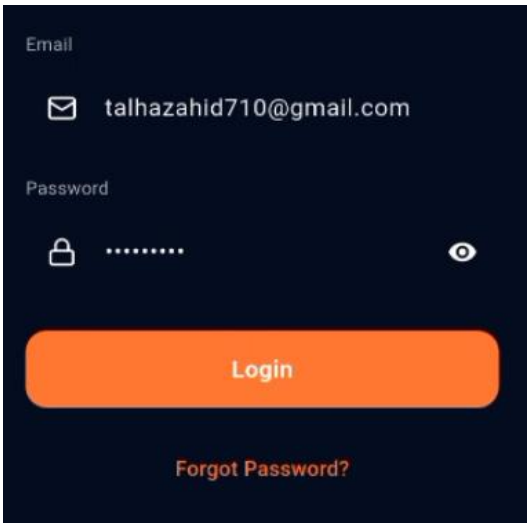
Here are the test scenarios we will cover:

- Verify user can log in with valid credentials.
- Verify user cannot log in with invalid credentials.
- Verify error message is displayed for invalid login attempts.
- Verify users can sign up with valid information.

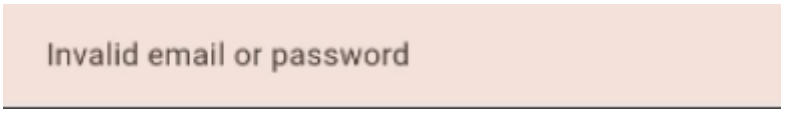
- Verify users cannot sign up with incomplete information.
- Verify email verification is sent after sign-up.
- Verify users can request a password reset.
- Verify email is sent with password reset instructions.
- Verify user can reset the password using the provided link.
- Verify user receives an email verification link after sign-up.
- Verify users can verify their email using the link.
- Verify error message is shown for expired or invalid verification links.
- Verify tourists can view and update their profile.
- Verify tourists can access the home page and see popular places, hotels, and recommended plans.
- Verify tourists can search for places and filter results.
- Verify tourists can view details of a specific place and leave ratings and reviews.
- Verify tourists can like a place and see it in their liked places.
- Verify tourist can search for hotels, filter results, view hotel details, book hotels, and view bookings.
- Verify tourist can search for packages, filter results, view package details, book packages, and view bookings.
- Verify tourist can create a custom tour plan and download it as a PDF.
- Verify tourist receives email notifications for bookings and updates.
- Verify tourist can use the contact form to reach hotel managers and tour operators, and queries are acknowledged.
- Verify static pages load correctly and display the appropriate content.
- Verify admin functionalities for managing places, recommended plans, hotels, packages, and user accounts.
- Verify content manager functionalities for managing places and recommended plans.
- Verify hotel manager functionalities for managing hotels and viewing bookings.
- Verify tour operator functionalities for managing packages.

Test Cases

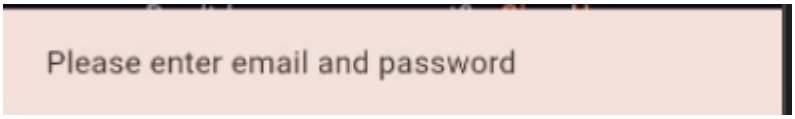
Test Case ID	TC_TOPK_01
Test Scenario	Login
Test Case description	With Valid Data Set
Steps to perform	1. Go login page 2. Enter the Email: talhazahid710@gmail.com 3. Enter Password: talha@130 4. Click submit button
Expected Result	The system should go to the home page.
Actual Result	As expected,
Test Status	Pass

Screen shot	

Test Case ID	TC_TOPK_02
Test Scenario	Login
Test Case description	With an Invalid Data Set

Steps to perform	1. Go login page 2. Enter the Email: ahmedmujtaba786kips@gmail.com 3. Enter Password: programmer@12345 4. Click submit button
Expected Result	The system should stop users from logging and landing on the home page.
Actual Result	The system stopped the user from logging and landing on the home page.
Test Status	Pass
Screenshot	

Test Case ID	TC_TOPK_03
Test Scenario	Login error message
Test Case description	With Valid Data Set
Steps to perform	1. Go login page 2. Enter the Email: talhazahid710@gmail.com 3. Enter Password: talha@130 4. Click submit button
Expected Result	The system should stop the user from login and landing on the home page and also show the proper error message as per the field accordingly
Actual Result	As expected,
Test Status	Pass

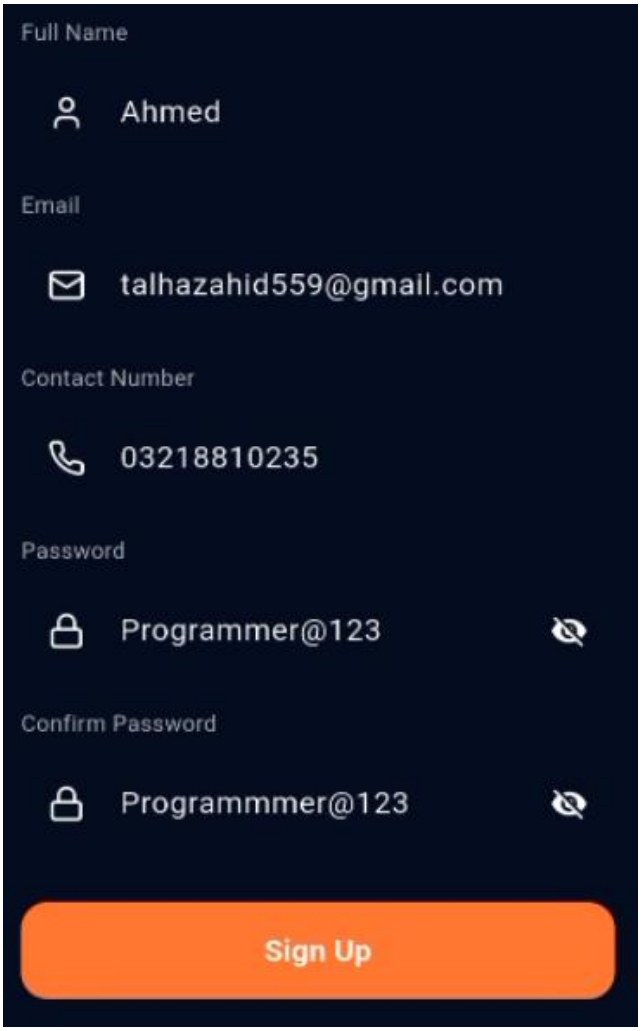
Screenshot	

Test Case ID	TC_TOPK_04
Test Scenario	Login button
Test Case description	Login disable button after one time clicking on it
Steps to perform	1. Go login page 2. Enter the Email: ahmedmujtaba.cs@gmail.com 3. Enter Password: Programmer@123 4. Click submit button
Expected Result	The system should disable the login button after the user clicks on it.
Actual Result	The system disabled the login button after the user once clicked on it.
Test Status	Pass

Test Case ID	TC_TOPK_05
Test Scenario	Login Direct Linking
Test Case description	Direct link to admin home page without login
Steps to perform	Enter the URL of the admin home page in the URL bar.
Expected Result	The system should go back to the login page and show an error message
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_06
Test Scenario	Sign up
Test Case description	Test with all required fields with the correct values

Steps to perform	1. Go sign up page 2. Enter Name: Ahmed 3. Enter the Email: talhazahid559@gmail.com 4. Enter Phone: 03218810235 5. Enter CNIC: 35200-3255000-2 6. Enter Password: Programmer@123 7. Enter Confirm Password: Programmer@123 8. Click the submit button

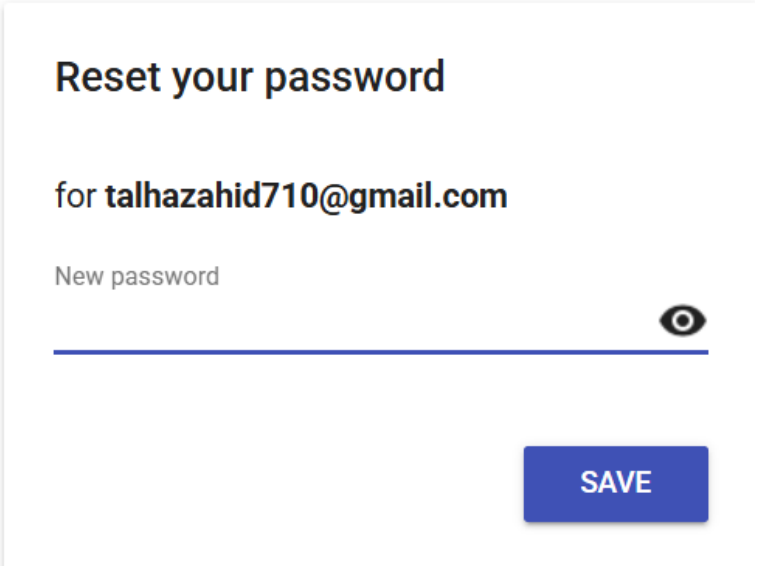
Expected Result	The system should go to the Email Verification page and the User should receive the OTP on entering email, After Verifying the OTP, the System should successfully sign up the user data and go to the home page.
Actual Result	As expected,
Test Status	Pass
Screen shot	

Test Case ID	TC_TOPK_07
Test Scenario	Sign up
Test Case description	Test with all required fields with the wrong values
Steps to perform	1. Go sign up page 2. Enter Name: Ahmed 3. Enter the Email: talhazahid559@gmail.com 4. Enter Phone: 03218810235776 5. Enter CNIC: 3520032550241 6. Enter Password: programmer123

	7. Enter Confirm Password: programmer123 8. Click the submit button
Expected Result	The system should stop the user from signing up and show a proper error message, the email should have the proper format, CNIC must have a dash in the format, the phone number must be 11 digits, the password must contain one capital letter, one special character, and one digit and Password and confirm password must be same in both field.
Actual Result	As expected,
Test Status	Pass

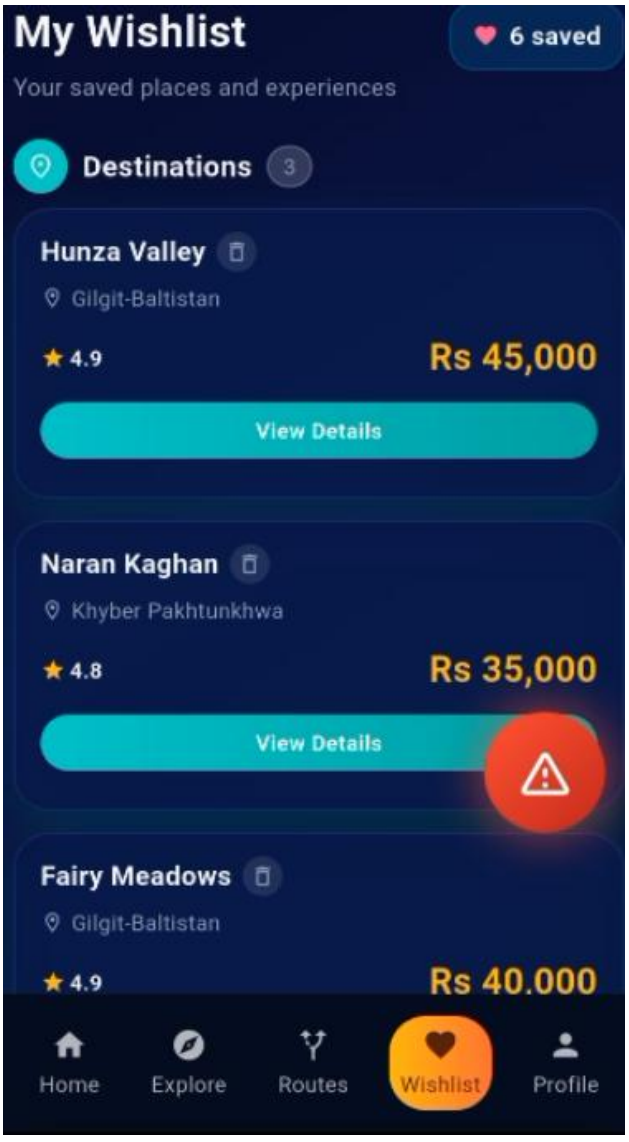
Test Case ID	TC_TOPK_08
Test Scenario	Sign up
Test Case description	Signup with the existing user email, phone, CNIC
Steps to perform	1. Go sign up page 2. Enter Name: Ahmed 3. Enter the Email: talhazahid559@gmail.com 4. Enter Phone: 03218810235 5. Enter CNIC: 35200-3255024-1 6. Enter Password: Programmer@123 7. Enter Confirm Password: Programmer@123 8. Click the submit button
Expected Result	The system should stop the user from signing up and show a proper error message, the email, CNIC, and Phone are already registered.

Actual Result	As expected,
Test Status	Pass

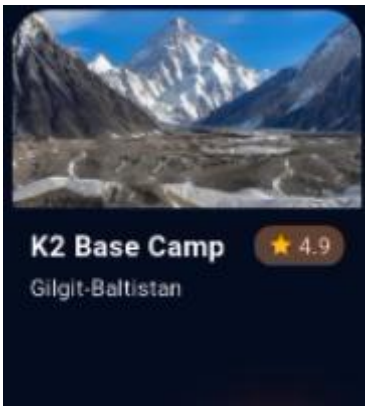
Test Case ID	TC_TOPK_09
Test Scenario	Forgot Password
Test Case description	The system should allow the user to forget their password.
Steps to perform	1. Go Login Page Click on Forgot Password button 2. Enter the PIN code received on the registered email 3. Enter New Password 4. Enter Confirm Password 5. Click on the submit button
Expected Result	The system should go to the forgotten password page, user should receive a PIN code on the registered email, after verifying the PIN code, the system should go to the forgotten password page, user should be able to enter a New strong password and land on the login page.
Actual Result	As expected,
Test Status	Pass
Screen shot	

Test Case ID	TC_TOPK_010
Test Scenario	Tourist Profile
Test Case description	Verify users can view and update their profile information.
Steps to perform	1. Go to the home page of TourToPK 2. Click on the Profile button 3. View and update their information 4. Click on the save change button
Expected Result	The system allows verified users to view and make changes in name, email, CNIC, phone, home town, and picture in their profile section.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_011
Test Scenario	Search and Filter for places
Test Case description	Test for searching and filter the places with title, province, and rating.
Steps to perform	1. Go to the Places Listing page 2. Search by place title or by entering a city name 3. Apply filters by Province, Place Type, and Order by rating and liked places.
Expected Result	The system should show only the filtered places and places that are searched by users.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_012
Test Scenario	Liked places
Test Case description	Verified user able to like a place.
Steps to perform	1. Go to the Places Listing page 2. Click the heart icon on the Place card 3. Click on the “Are you sure” button on the popup
Expected Result	The system should allow the verified user to add places in their liked places section and also allow unliked places from the liked places section
Actual Result	As expected,
Test Status	Pass
Screen shot	

Test Case ID	TC_TOPK_013

Test Scenario	Ratings and Reviews of places
Test Case description	Verified user able to give a rating and review to places
Steps to perform	1. Go to the Places Listing page 2. Click on the Places card 3. Choose a Rating star 4. Write a review in the comment section 5. Click on the submit button
Expected Result	The system should allow verified users to give ratings and reviews places.
Actual Result	As expected,
Test Status	Pass
Screen shot	

Test Case ID	TC_TOPK_014
Test Scenario	Search and Filter for Hotels
Test Case description	Test for searching and filtering the hotels with title, province, price, and rating.
Steps to perform	1. Go to the hotel Listing page 2. Search by hotel name 3. Apply filters by Province, Price Range, and Order by rating and expensive, cheapest hotel.
Expected Result	The system should show only the filtered hotels and hotels that are searched by users.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_017
Test Scenario	Custom Tour Plan
Test Case description	Verified users can make their custom tour plan and download it in PDF format.
Steps to perform	1. Click on the Custom Tour butt 2. Fill out your Personal information section 3. Enter Full name: Ahmad 4. Choose City: Lahore 5. Enter Contact Number: 03218810235 6. Choose Occupation: Student 7. Enter Email: talhazahid559@gmail.com 8. Enter Confirm Email: talhazahid559@gmail.com 9. Fill out the Tour informational section 10. Choose your preference: Standard 11. Enter Budge: 80,000/- 12. Enter start date: 05/06/2024 13. Enter end date: 08/06/2024 14. Enter Visitors: 5 15. Enter adults: 5 16. Enter children: 0 17. Enter infants: 0 18. Fill out the destination section 19. Choose the tour location 20. Select multiple places 21. Click on the submit button 22. Click on the download button

Expected Result	The system should allow verified users to make their custom tour plan and download it in PDF format. All required fields must be valid in standard format respectively.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_018
Test Scenario	User Preference
Test Case description	Verified user able to give place suggestions, not present in place listing.
Steps to perform	1. Go to the Places Listing page 2. Click the Suggest Us button 3. Enter the Place title 4. Enter description 5. Enter activities to do 6. Enter Famous food to eat 7. Click on the submit button
Expected Result	The system should allow verified users to suggest the places. TourToPK Content Manager can view and delete user preferences.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_019
Test Scenario	Search and Filter for Recommended tour plans
Test Case description	Test for searching and filter the recommended plans with title and province.
Steps to perform	1. Go to the Recommended Plan Listing page 2. Search by Recommended Plan title 3. Apply filters by Province and by number of days
Expected Result	The system should show only the filtered plans and plans that users search.

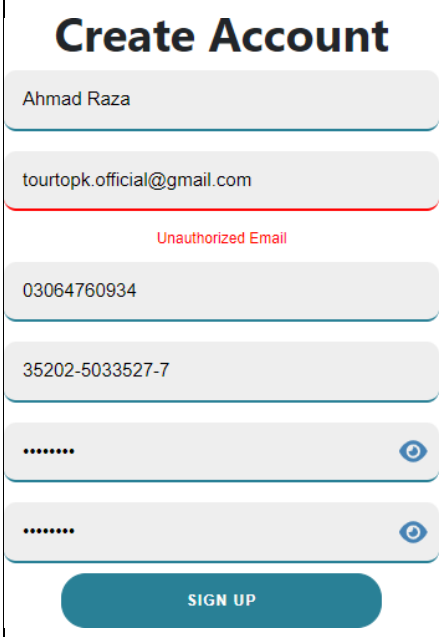
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_020
Test Scenario	Search and Filter for Packages
Test Case description	Test for searching and filter the Packages with title and Province.
Steps to perform	1. Go to the Packages Listing page 2. Search by Packages title.
	3. Apply filters by Province, and Order by rating to Packages
Expected Result	The system should show only the filtered Packages and Packages that users search.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_021
Test Scenario	Ratings and Reviews of Packages
Test Case description	Verified user able to give a rating and review Packages
Steps to perform	1. Go to the Packages Listing page 2. Click on the Packages card 3. Choose a Rating star 4. Write a review in the comment section 5. Click on the submit button
Expected Result	The system should allow verified users to give ratings and reviews Packages
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_022
Test Scenario	Contact Us for partner's accounts
Test Case description	Tourists, hotel managers, and tour operators can email to Admin for queries.
Steps to perform	1. Go to the Contact Us page 2. Enter Full Name: Ahmad 3. Enter Email: ahmedmujtaba942kips@gmail.com 4. Enter Subject: how to register my hotel 5. Enter Message: Hello, I want to register my hotel…. 6. Click on the send email button
Expected Result	The system should allow Tourists, hotel managers, and tour operators to send emails for account creation and any other queries.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_023
Test Scenario	Official Email
Test Case description	Test to verify TourToPK official Email cannot be used as a Tourist.
Steps to perform	1. Go sign up page 2. Enter Name: Ahmad 3. Enter the Email: tourtopk.official@gmail.com 4. Enter Phone: 03218810235 5. Enter CNIC: 35200-3255024-1 6. Enter Password: Programmer@123 7. Enter Confirm Password: Programmer@123 8. Click submit button
Expected Result	The system should stop users from signing up and show a proper error message that is an unauthorized email.
Actual Result	As expected,
Test Status	Pass

Screenshot	

Test Case ID	TC_TOPK_024
Test Scenario	TourToPK Admin home
Test Case description	Test Super Admin can view and delete places, recommended plans, hotels, and packages.
Steps to perform	1. Go to the login Page 2. Enter Username: Admin 3. Enter password: ********* 4. Click on Sign-in button 5. Click on the Places button 6. Click on the Plans button 7. Click on the Hotels button 8. Click on the Packages button
Expected Result	The system should allow the Admin to View and delete Places, Recommended Plans, Hotels, and Packages. Also, the admin can search for these respected features.

Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_025
Test Scenario	Partner Account creation.
Test Case description	Test Super Admin should be able to Create partner accounts
Steps to perform	1. Go to the login Page 2. Enter Username: Admin 3. Enter password: ********* 4. Click on Sign-in button 5. Click on the Add Partners Button 6. Enter Company Name: 7. Enter CNIC: 8. Enter Email: 9. Enter Phone: 10. Choose Partner Type: 11. Enter 4 digit random number 12. Choose Province: 13. Choose City: 14. Enter Location: 15. Click on submit
Expected Result	The system should allow the super admin to create hotel manager or tour operator accounts, CNIC should be unique and contain dashes, the same email cannot be used for TO or HM, the user name should be unique, the Phone number must contain 11 digits, Cities must correlate with the respective province.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_026
Test Scenario	Test Content Manager Manages Places
Test Case description	Content managers should be able to Add, Update, Delete, View Places
Steps to perform	1. Go to the login Page 2. Enter Email: manager@tourtopk.com 3. Enter password: ********* 4. Click on Sign-in button 5. Click on the Add Places button 6. Enter place title: 7. Enter a description: 8. Enter activities: 9. Enter famous food: 10. Choose Province: 11. Choose City: 12. Enter Location: 13. Choose Place Type: 14. Choose images: 15. Click on submit To view, delete, and update click on the View Places button
Expected Result	The system should allow the content manager to add places, after submitting places details page should navigate to the all places page where the content manager can view, update, and delete places.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_027
Test Scenario	Test Content Manager Manages Recommended Plans
Test Case description	Content managers should be able to Add, Update, Delete, View Recommended Plans
Steps to perform	1. Go to the login Page 2. Enter Email: manager@tourtopk.com 3. Enter password: ********* 4. Click on Sign-in button 5. Click on the Add Plans button 6. Enter Plans title: 7. Enter Description: 8. Choose Province: 9. Select the number of days: 10. Select Places according to days: 11. Choose images: 12. Click on submit To view, delete, and update click on the View Plans button
Expected Result	The system should allow the content manager to add plans, after submitting the recommended plans details page should navigate to the all recommended plan page where the content manager can view, update, and delete plans. Places must be according to the respective province. Allow only 3,5,7 number of days, only a maximum 5 number of images.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_028
Test Scenario	User Preference for Content Manager
Test Case description	Test content manager able to view and delete user preferences.

Steps to perform	1. Go to the login Page 2. Enter Email: manager@tourtopk.com 3. Enter password: ********* 4. Click on Sign-in button 5. Click on the delete icon to delete user preference 6. Click on the confirm button
Expected Result	The system should allow the content manager to view and delete user preferences.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_029
Test Scenario	Hotel Manager Profile
Test Case description	Hotel managers can view and update their profile information.
Steps to perform	1. Go to the hotel manager's dashboard home 2. Click on the Profile button 3. View and update their information

	4. Click on the save change button
Expected Result	The system allows hotel managers to view and make changes in name, email, CNIC, phone, home town, and picture in their profile section.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_030
Test Scenario	The test hotel manager manages hotel details.
Test Case description	Verify hotel manager can create, view, update, and delete hotels.

Steps to perform	1. Go to the hotel manager's dashboard home 2. Click on the add hotel button 3. Enter hotel name: 4. Choose amenities: 5. Enter a description: 6. Enter the number of beds: 7. Enter price per night: 8. Enter the number of bathrooms: 9. Enter the number of guests: 10. Choose images: 11. Click on submit To view, delete, and update click on the View Hotels button
Expected Result	The system should allow the verified hotel manager to add a hotel, after submitting the hotel details page should navigate to the all hotel page where the hotel manager can view, update, and delete hotels. The hotel name should be alphabets, number of beds, number of bathrooms, number of guests, price should be a valid positive number. Otherwise, systems should show a proper error message.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_031
Test Scenario	Booking details to the hotel manager
Test Case description	Verify hotel manager can view hotel bookings.
Steps to perform	1. Go to the hotel manager's dashboard home 2. Click on the booking button 3. Click on the delete icon to delete the booking.

	4. Click on submit To view, delete, and update click on the View Hotels button
Expected Result	The system should allow the hotel manager to view and delete tourist bookings.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_032
Test Scenario	Tour Operator Profile
Test Case description	Tour operators can view and update their profile information.
Steps to perform	1. Go to the tour operator dashboard home 2. Click on the Profile button 3. View and update their information 4. Click on the save change button
Expected Result	The system allows tour operator to view and make changes in name, email, CNIC, phone, home town, and picture in their profile section.
Actual Result	As expected,
Test Status	Pass

Test Case ID	TC_TOPK_033
Test Scenario	Test tour operator manages packages
Test Case description	Verify tour operator can create, view, update, and delete packages
Steps to perform	1. Go to the tour operator dashboard home 2. Click on the add package button 3. Enter Package title: 4. Enter a description: 5. Enter the content number: 6. Enter price: 7. Enter the dispatch date: 8. Enter the dispatch city: 9. Enter hotel name: 10. Choose famous food: 11. Choose activities:

	12. Select province: 13. Select city: 14. Choose images: 15. Click on submit

Expected Result	The system should allow the verified tour operator to add a package, after submitting the package details page should navigate to the all packages page where the tour operator can view, update, and delete packages. The package name should be alphanumeric, the contact number should be 11 digits, a maximum of 5 images will be uploaded, price, dispatch date, and city, famous food, and activities should be valid according to format. Cities should be correlated with the respective province. Otherwise, systems should show a proper error message.
Actual Result	As expected,
Test Status	Pass

Appendices

Appendix A: Information / Promotional Material

This appendix presents the informational and promotional material developed for the SafarSync mobile application. It includes structured content used to communicate the app's features, benefits, and usage to potential users and stakeholders. The material is designed to enhance awareness, improve user engagement, and support the overall marketing strategy. This appendix complements the main report by providing supporting content without interrupting the technical discussion.

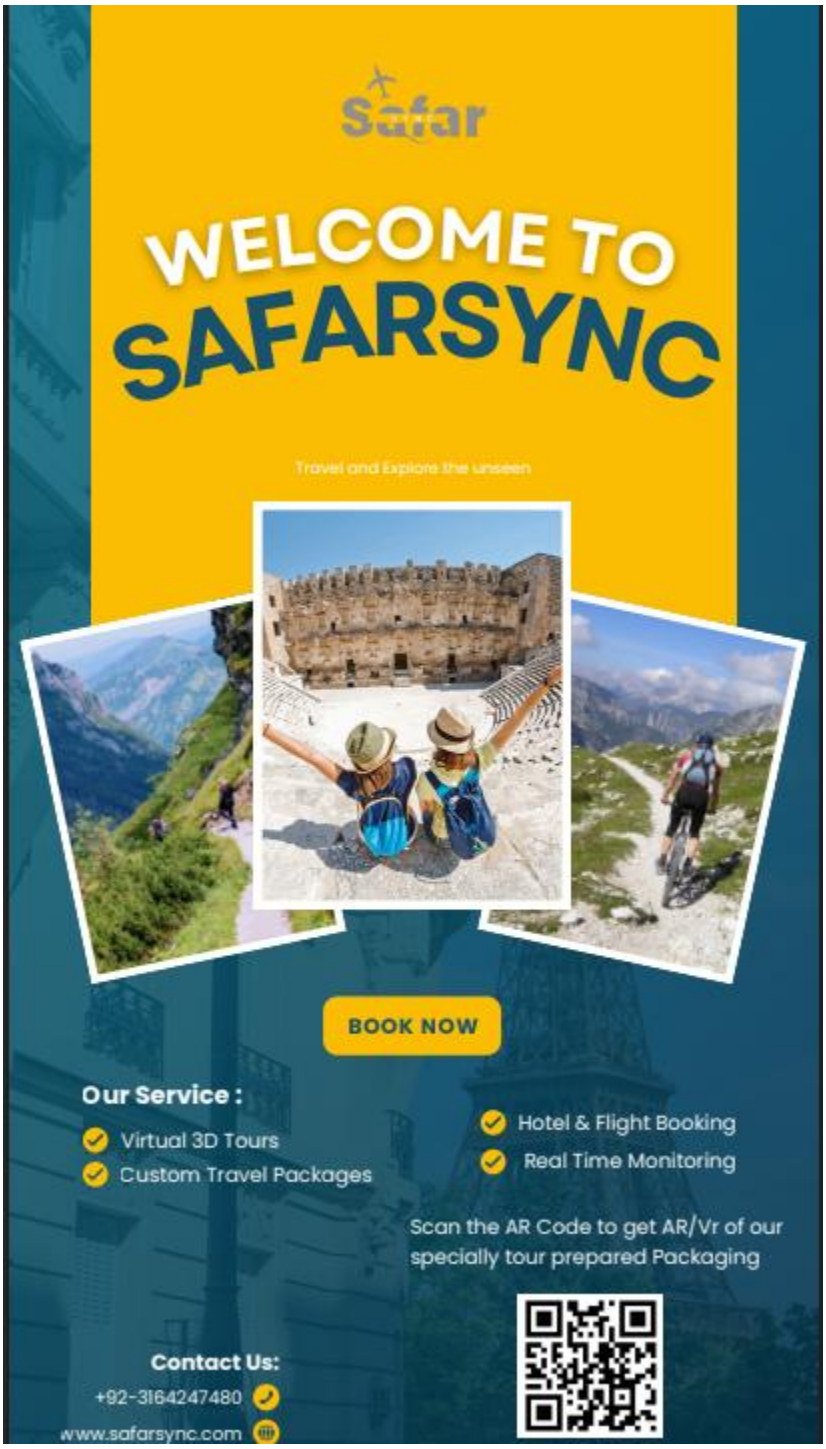
A.1. Broacher



A.2. Flyer


www.safarsyncc om
Safar
Explore the unseen
Get Virtual Tours, Advance Booking and many more
RUSSIA
KAZAKHSTAN
USA
OCEAN

A.3. Standee


Safar
WELCOME TO
SAFARSYNC
Travel and Explore the Unseen
BOOK NOW
Our Service :
Virtual 3D Tours
Custom Travel Packages
Hotel & Flight Booking
Real Time Monitoring
Scan the AR Code to get AR/Vr of our specially tour prepared Packaging
Contact Us:
+92-3164247480
www.safarsync.com

A.4. Banner


IT'S TIME TO
TRAVEL
Safar
BURZIL TOP
HT 13808 Ft
KARGIL HEROES
PACKAGE 1
3 Days, 2 Nights Naran Tour
Room, Flight, Bus Booking
Cultural/Historical Points Coverage
14,000/PERSON
PACKAGE 2
5 Days, 4 Nights Astore Tour
Room, Flight, Bus Booking
3D Virtual Tour
ICE Support
20,000/PERSON
PACKAGE 3
7 Days Hunza Tour
Room, Flight, Bus Booking
3D Virtual Tour
ICE Support
35,000/PERSON
Call for more
0316-4247480
Book Now!
Lahore,
Pakistan

A.5. Promotional Overview

This section provides an overview of the promotional approach adopted for the SafarSync application. The promotional strategy focuses on highlighting key features such as smart route planning, emergency assistance, and personalized recommendations. The aim is to attract users by clearly communicating the value and usability of the system through visually engaging and informative content.

A.5.1. Target Audience and Objectives

The primary target audience for SafarSync includes travelers, tourists, and daily commuters seeking smart and safe navigation solutions. The promotional material is designed to inform users about the system's capabilities while encouraging app adoption. Key objectives include increasing user awareness, building trust, and improving overall user engagement.

A.1.1.1. Promotional Content Description

The promotional content includes application screenshots, feature highlights, and short descriptive texts explaining core functionalities. Visual elements are designed with a modern and user-friendly layout to ensure clarity and appeal. The content emphasizes simplicity, safety, and smart travel assistance to effectively communicate the system's benefits.

A.1.1.2. Distribution Platforms

The promotional material is intended for distribution across multiple platforms, including mobile application onboarding screens, social media channels, and project presentations. This multi-platform approach ensures maximum visibility and accessibility for diverse user groups.